

Understanding Vision Transformers through Implementation and Experimentation

Thadar Soe¹

¹Pusan National University

1. Introduction

This lab focused on implementing and understanding Vision Transformers (ViT)[2] through hands-on experimentation. The primary goal was to understand the internal mechanisms of Vision Transformers by building each component step-by-step and visualizing intermediate outputs. I worked with a pre-trained ViT-Base model provided by the professor (patch size 16, image size 224) and implemented three critical components: patch embedding, position encoding integration, and attention mechanism computation. Through this process, I discovered how transformers, originally designed for sequential text processing, can be adapted to handle spatial image data. The experiments provided valuable insights into how the model divides images into patches, learns spatial relationships, and uses multi-head attention to focus on different image regions, leveraging the core transformer principles[4].

2. TODO 1: Implementing Patch Embedding

2.1. Implementation Process

The first TODO involved implementing a custom patch embedding module to understand how Vision Transformers convert images into sequences of patch embeddings. I implemented the `PatchEmbed` class, which divides a 224×224 image into $14 \times 14 = 196$ patches of size 16×16 each. The key challenge was understanding how to use a convolutional layer with a specific kernel size and stride to achieve this division:

```
class PatchEmbed(nn.Module):
    def __init__(self, img_size=224, patch_size=16,
                 in_chans=3, embed_dim=768):
        super(PatchEmbed, self).__init__()
        self.patch_size = patch_size
        self.grid_size = (img_size // patch_size,
                          // patch_size, img_size // patch_size)
        self.num_patches = self.grid_size[0] * self.grid_size[1]

        self.img_size = img_size
        self.proj = nn.Conv2d(
            in_chans,
            embed_dim,
            kernel_size=patch_size,
            stride=patch_size
        )
```

2.2. Key Findings

Through this implementation, I learned that using a Conv2D layer with both kernel size and stride set to 16 effectively divides the image into non-overlapping patches while simultaneously projecting them to the embedding dimension (768). The transformation sequence: $(B, 3, 224, 224) \rightarrow (B, 768, 14, 14) \rightarrow (B, 768, 196) \rightarrow (B, 196, 768)$ clarified how spatial dimensions are flattened into a sequence. By visualizing the 196 patches, I also confirmed how the model "sees" the image as a grid of small image regions rather than as a whole. During this process, I initially found the purpose of the transpose operation confusing, but I eventually understood that the final transpose $(B, 768, 196) \rightarrow (B, 196, 768)$ is necessary because transformers expect the sequence dimension before the feature dimension.

3. TODO 2: Position Embedding Analysis

The second TODO focused on understanding how position embeddings help the model maintain spatial awareness when processing image patches as a 1D sequence. I calculated the cosine similarity between each patch position embedding and all other position embeddings as follows:

```
for i in range(1, pos_embed.shape[1]):
    current_patch = pos_embed[0, i:i+1, :]
    all_patches = pos_embed[0, 1:, :]
    sim = cos(current_patch, all_patches)
    sim = sim.reshape(14, 14)
```

This computation produced a 14×14 similarity matrix for each patch position, which I visualized as heatmaps. This visualization revealed several important patterns:

- Patches closer in spatial distance had higher cosine similarity values.
- The similarity gradually decreased with increasing distance, forming a smooth gradient.
- Even though patches were processed as a 1D sequence, I observed that the position embeddings learned a 2D spatial structure.
- Each position embedding uniquely encoded its location in the image grid.

4. TODO 3: Attention Mechanism Computation

For the third experiment, I implemented the attention matrix calculation to understand how the model determines which parts of the image to focus on. I applied the scaled dot-product attention formula:

```
d_k = q.shape[-1] # 64
attention_matrix = torch.matmul(q, kT) / sqrt(d_k)
attention_matrix = F.softmax(attention_matrix, dim=-1)
```

Here, q has shape $(H = 12, N = 197, D/H = 64)$ and kT has shape $(H = 12, D/H = 64, N = 197)$. The resulting attention matrix with shape $(12, 197, 197)$ represents 12 attention heads, each computing attention scores between all 197 tokens (1 CLS + 196 patches). The scaling factor $\sqrt{d_k}$ prevents the dot products from becoming too large, which would cause the softmax to produce extremely small gradients. The softmax operation ensures that attention weights sum to 1 across each row, effectively representing a probability distribution over which tokens to attend to.

5. Image Classification Experiment

5.1. Testing the Complete Pipeline

I tested the complete Vision Transformer pipeline on a sample image of Santorini, Greece. The model successfully classified it as "church, church_building" (class ID 497 out of 1000 ImageNet classes).

5.2. Debugging Process

An important lesson I learned during this experiment was understanding the difference between my custom implementation and the pre-trained model:

- **Initial Error:** Using `my_patches` from my custom `PatchEmbed` resulted in incorrect classification ("wall_clock", class ID 892).

- **Solution:** Switching to `model.patch_embed(img_tensor)` ensured the use of pre-trained weights, which produced the correct classification.
- **Lesson:** When performing inference with pre-trained models, all components must use the learned weights rather than randomly initialized custom modules. This reinforces the finding that ViTs often require large-scale pre-training or data-efficient techniques like distillation[3].

6. Conclusion

This hands-on lab allowed me to gain a deeper understanding of Vision Transformers by implementing core components and analyzing their internal behaviors. By building patch embedding, integrating position encodings, and computing attention matrices from scratch, I gained practical insights that extend beyond theoretical knowledge.

The experiments demonstrated how Vision Transformers process images fundamentally differently from CNNs—treating images as sequences of patches with learned spatial relationships and using multi-head attention to capture diverse features. Visualizing position embeddings and attention patterns made abstract concepts more concrete and interpretable. Analyzing attention patterns in depth could further reveal how self-attention leads to emergent semantic properties[1]. Additionally, the debugging process behind achieving correct classification emphasized the importance of using pre-trained weights properly while still understanding how each component works internally. This balance between understanding and application is essential for effectively working with modern deep learning architectures.

Future extensions of this work could involve analyzing deeper layers of the ViT model, comparing attention behaviors across different images, or implementing additional transformer components such as the MLP blocks within the encoder.

- [1] Mathilde Caron, Ishan Misra, Julien Mairal, Priya Goyal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *International Conference on Computer Vision (ICCV)*, pages 4680–4690, 2021.
- [2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jacob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://arxiv.org/abs/2010.11929>.
- [3] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Synnaeve. Training data-efficient vision transformers knowledge distillation in vision. In *International Conference on Machine Learning (ICML)*, pages 10347–10357, 2021.
- [4] Ashish Vaswani, Niki Shazeer, Nitesh Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.